

Linux_Wk4/S3 Lecture #11

Topics Covered:-

1. Vim Modes, Insert Mode, Normal Mode
2. Vim Editing, Saving, Exiting, Searching, Movement
3. Linux File Access Basics
4. Read, Write, Execute Permissions
5. User, Group, Others
6. Permission String (ls -l)
7. Symbolic chmod
8. Numeric chmod (777, 755, 600)
9. File Ownership Concepts
10. chown (user, group, recursive, advanced)

Linux File Viewing & Editing

Text Editor: vim

1. What Vim Is

Vim is **not** like nano or Windows editors.

In Vim:

- Keys do **different things at different times**
- This is because Vim uses **modes**

If you do not understand modes, Vim feels broken.
Once you understand modes, Vim becomes predictable.

2. The Single Most Important Rule of Vim

Vim is always in one mode at a time

There are many modes, but beginners only need **two** to start:

1. **Normal Mode** → for commands
 2. **Insert Mode** → for typing text
-

3. Start Vim

Open a terminal and run:

```
vim test.txt
```

What you see:

- A mostly empty screen
- Possibly some ~ symbols on the left
- No menu, no mouse cursor

You are now in **NORMAL MODE**

4. How to Know Which Mode You Are In

Look at the **bottom-left of the screen**.

If you see:

-- INSERT --

- You are in **Insert Mode**
- If you see **nothing like that**, you are in **Normal Mode**

This visual indicator is critical.

6. Correct Way to Enter Insert Mode

Step 1: Ensure Normal Mode

Press:

Esc

(Press it even if you think you're already there — it's safe.)

Step 2: Enter Insert Mode

Press:

i

Step 3: Confirm Insert Mode

Look at the screen bottom:

-- INSERT --

If you see this → **success**

7. Editing

Step 1:

`vim hello.txt`

Step 2: Reset Mode

Press:

Esc

(This guarantees Normal Mode.)

Step 3: Enter Insert Mode

Press:

i

Confirm you see:

-- INSERT --

Step 4: Type Text

Now type **exactly this**:

Hello World

This is my first Vim file

Step 5: Exit Insert Mode

Press:

Esc

Text stops appearing when you type — this is correct.

8. Saving the File

You must be in **Normal Mode**.

Step 1: Press

:

The cursor moves to the bottom.

Step 2: Type

w

Step 3: Press

Enter

The file is now saved.

9. Exiting Vim Safely

Again, ensure **Normal Mode**:

Press:

Esc

Then:

:q

Press:

Enter

Vim closes.

10. Save and Exit Together

Instead of separate save and quit:

:wq

Then press:

Enter

11. If Vim Refuses to Exit

If Vim shows an error like:

E37: No write since last change

This means:

- You edited the file
- You didn't save

To force exit (lose changes):

`:q!`

Press:

Enter

12. Golden Recovery Rule

When Vim behaves strangely:

1. Press Esc
 2. You are safe again
-

13. Moving Around (Normal Mode Only)

Press Esc first.

Key	Action
h	Move left
j	Move down
k	Move up
l	Move right

Arrow keys also work, but these are standard.

14. Adding More Text Correctly

You must **re-enter Insert Mode**.

Steps:

1. Esc
2. i
3. Type text
4. Esc

Repeat this cycle as needed.

15. Deleting Text (Normal Mode)

Command	Effect
x	Delete one character
dd	Delete one line

If text deletes instead of typing — you are **not** in Insert Mode.

16. Searching Text

Ensure Normal Mode (Esc).

Press:

/word

Press:

Enter

Navigate:

- n → next match
 - N → previous match
-

17. Opening File at Specific Line

`vim +10 file.txt`

Cursor opens at line 10.

18. Read-Only Mode

`vim -R important.conf`

You can view but not edit.

Final Truth About Vim

- Vim is **not broken**
 - It is **strict**
 - Once you control modes, Vim obeys perfectly
-

LINUX FILE PERMISSIONS

1. How Linux Thinks About File Access

Linux is a **multi-user operating system**.

This means:

- Many users can exist on one machine.
- Every file must define:
 - **Who** can access it
 - **What** they are allowed to do

Linux solves this using:

- **Ownership**
- **Permissions**

Before learning commands like `chmod`, it is essential to understand permissions deeply.

2. Three Basic Permissions — The Heart of Everything

Linux uses only **three permission types**.

These must be understood first because everything later depends on them.

2.1 Read Permission — **r**

For files, **read** means:

- ✓ View content
- ✓ Open the file
- ✓ Use commands such as:

```
cat demo.txt  
less demo.txt
```

Example

Create a file:

```
touch demo.txt  
echo "Hello Students" > demo.txt
```

Read the file:

```
cat demo.txt
```

Expected output:

Hello Students

Explanation:

Reading works because read permission exists.

2.2 Write Permission — w

Write permission allows:

- ✓ Editing content
- ✓ Adding text
- ✓ Removing text

Practical Example

```
echo "New Line" >> demo.txt
```

Check:

```
cat demo.txt
```

Output:

Hello Students
New Line

Explanation:

Write permission allows modification.

2.3 Execute Permission — x

This is where beginners often get confused.

Execute does **not** mean open.

Execute means:

Run a file like a program.

command:

./demo.txt

Expected result: **Permission denied**

Explanation:

Execution fails because execute permission is missing.

3. Who Gets These Permissions?

Linux divides users into three categories.

Always remember this order:

USER GROUP OTHERS

3.1 User (Owner)

The person who created the file.

Example system user:

prabhat

3.2 Group

A collection of users, such as:

- developers
- students
- admins

Group permissions allow teamwork.

3.3 Others

Everyone else on the system, including:

- user2
- any other login

4. First Look at Permissions Using `ls -l`

Now that r, w, x and user types are understood:

`ls -l demo.txt`

Example output:

`-rw-r--r-- 1 prabhat prabhat 20 demo.txt`

Focus on the first section:

`-rw-r--r--`

5. The 10-Character Permission String

`-rw-r--r--`

Split visually:

```
-  rw-  r--  r--
|  |  |  |
|  |  |  +--- Others permissions
|  |  +----- Group permissions
|  +----- User permissions
+----- File type
```

5.1 First Character — File Type

1. - means normal file.
2. d- it means directory.
3. l- link

5.2 User Permissions — rw-

Owner can:

- ✓ Read
- ✓ Write
- ✗ Execute

5.3 Group Permissions — r--

Group can only read.

5.4 Others Permissions — r--

Other users can only read.

6. Multi-User Demo

Create a second user:

```
sudo adduser user2
```

Switch to user2:

```
su - user2  
whoami
```

Output: **user2**

Go to original user folder:

```
cd /home/prabhat
```

If access is denied, allow folder access as prabhat:

```
chmod o+x /home/prabhat
```

Check permissions:

```
ls -l demo.txt
```

Still: **-rw-r--r--**

7. Tests as user2 (OTHERS)

Others permission = r--.

Read Test

cat demo.txt

Works.

Explanation: Others have read permission.

Write Test

echo "user2 edit" >> demo.txt

Output:

Permission denied

Explanation: Others lack write permission.

Execute Test

./demo.txt

Output:

Permission denied

Explanation: Others lack execute permission.

INTRODUCING chmod

Current state:

File = demo.txt
Owner = prabhat
Second user = user2
Permission = -rw-r--r--

1. Why We Need chmod

Until now, permissions were only observed.

We saw:

Action	Owner	user2
Read	✓	✓
Write	✓	✗
Execute	✗	✗

To change permissions, Linux uses:

chmod

Meaning:

change mode (change permissions)

2. Understanding chmod Language

chmod works using three pieces:

WHO ACTION PERMISSION

3. WHO — Which Users Are We Changing?

Symbol	Meaning
u	user (owner)
g	group
o	others
a	all (u+g+o)

4. ACTION — What Are We Doing?

Symbol	Meaning	Idea
+	Add permission	Give extra power
-	Remove permission	Take power away
=	Set exact permission	Replace everything

Important:

- + does not remove old permissions
- - removes specific permission
- = overwrites existing permissions

5. PERMISSION — Which Access?

Letter	Meaning
r	read
w	write
x	execute

6. Full Structure of Symbolic chmod

chmod WHO ACTION PERMISSION filename

Example:

chmod u+w demo.txt

7. Check Current State

Switch to owner:

whoami

ls -l demo.txt

Expected:

-rw-r--r--

8. First Real Change — Remove Write from Owner

Before change:

echo "Owner can edit" >> demo.txt

Works.

Run:

chmod u-w demo.txt

Check:

ls -l demo.txt

Expected: **-r--r--r--**

Proof:

echo "This should fail" >> demo.txt

Output: Permission denied

9. Second Change — Add Execute Permission

Test first:

```
./demo.txt
```

Output:

Permission denied

Run:

```
chmod u+x demo.txt
```

Check:

```
-r-xr--r--
```

Execute is added without changing other permissions.

10. Third Change — Add Execute for Everyone

```
chmod a+x demo.txt
```

Check:

```
-r-xr-xr-x
```

Everyone now has execute permission.

11. Fourth Change — Using "=" (Exact Permission)

```
chmod u=rwx,g=rx,o=r demo.txt
```

Breakdown:

- u=rwx → owner full access
- g=rx → group read + execute
- o=r → others read only

Result:

```
-rwxr-xr--
```

Test as user2

```
su - user2  
cd /home/prabhat  
cat demo.txt
```

Works.

```
echo "user2 edit" >> demo.txt
```

Fails.

Explanation: Others only have read permission.

12. Important Comparison

Symbol	Behaviour
+	Adds permission
-	Removes permission
=	Replaces permission

NUMERIC chmod (777, 755, 600)

Assume current file: **-rwxr-xr--**

Owner = prabhat

Second user = user2

1. Why Numeric chmod Exists

Symbolic commands are detailed, but admins often use numbers because numeric mode is faster.

Linux converts permissions into numbers.

2. Permission Values

Permission	Value
r	4
w	2
x	1

Linux **adds values**, not letters.

3. Building Numbers

$rwx = 4 + 2 + 1 = 7$

$rw- = 4 + 2 = 6$

$r-x = 4 + 1 = 5$

$r-- = 4$

$-wx = 2 + 1 = 3$

4. Why Three Numbers?

Linux uses:

user group others

So numeric chmod uses:

XYZ

Position	Meaning
X	user
Y	group
Z	others

Example:

755 = rwx r-x r-x

5. Example — chmod 777

Check current permissions:

ls -l demo.txt

As user2, writing fails:

echo "user2 test" >> demo.txt

Switch back to owner:

chmod 777 demo.txt

Result: **-rwxrwxrwx**

Test again as user2:

echo "user2 full access" >> demo.txt
cat demo.txt

Works.

Explanation: 777 gives write permission to everyone.

6. Example — chmod 755

```
chmod 755 demo.txt
```

Result:

```
-rwxr-xr-x
```

Test as user2:

```
echo "should fail" >> demo.txt
```

Output:

Permission denied

Others lost write permission.

7. Example — chmod 600

```
chmod 600 demo.txt
```

Result:

```
-rw-----
```

Only owner has access.

Test as user2:

```
cat demo.txt
```

Output:

Permission denied

8. Very Important Comparison

Command	Behaviour
chmod u+x	adds execute only
chmod 755	replaces all permissions
chmod 777	gives full access
chmod 600	private file

9. Questions

Q1. Why is chmod 777 dangerous?

Because anyone can:

- edit
 - delete
 - run malicious code
-

Q2. Why do admins prefer 755?

Because:

- owner = full control
- others = read + execute only

Safer than 777.

Q3. Does chmod 755 add permissions?

No.

Numeric chmod always **replaces** permissions.

LINUX FILE OWNERSHIP — chown

1. How Linux Thinks About Ownership

In Linux, every file has:

- An **owner (user)**
- A **group**

Ownership decides:

- Who controls the file
- Who can change permissions
- Who can transfer access

Before changing ownership using chown, we must understand how ownership appears in the system.

2. Understanding Ownership Using `ls -l`

Create a demo file:

```
touch demo.txt
ls -l demo.txt
```

Example output:

```
-rw-r--r-- 1 prabhat prabhat 0 demo.txt
```

Focus on these two columns:

```
prabhat prabhat
|      |
|      +---- group owner
+----- user owner
```

Explanation:

- First name = **file owner (user)**
- Second name = **group owner**

Linux stores both separately.

3. What Ownership Means in Real Behaviour

Owner has special power:

- ✓ Can change permissions
- ✓ Can modify file
- ✓ Can transfer ownership (with sudo/root)

Other users cannot change ownership of files they don't own.

4. Creating a Multi-User Environment

Create second user:

sudo adduser user2

Check current owner:

ls -l demo.txt

Example:

-rw-r--r-- 1 prabhat prabhat demo.txt

Switch user:

su - user2
whoami

Output: **user2**

Try editing ownership:

chown user2 demo.txt

Expected:

Operation not permitted

WHY:- Only root or privileged user can change ownership.

5. Why Ownership Matters Before Permissions

We often think permissions control everything.

But ownership controls:

- Who can run chmod
- Who can delete files
- Who can transfer access

So ownership sits **above permissions** in control hierarchy.

INTRODUCING chown

Current state:

File = demo.txt
Owner = prabhat
Group = prabhat
Second user = user2

1. What is chown?

chown means:

change owner

It allows changing:

- User owner
- Group owner
- Both together

2. Basic Structure of chown

sudo chown USER filename

Example:

sudo chown user2 demo.txt

Important:

Ownership change usually requires sudo/root.

3. First Practical Change — Change User Owner

Check current ownership:

ls -l demo.txt

Example:

-rw-r--r-- 1 prabhat prabhat demo.txt

Run:

sudo chown user2 demo.txt

Check again:

ls -l demo.txt

Expected:

-rw-r--r-- 1 user2 prabhat demo.txt

Explanation:

- User owner changed
- Group remained same

4. Switching Users to Prove Ownership Changed

Switch:

su - user2

Try editing file:

echo "Now I am owner" >> demo.txt

Works because user2 is now owner.

5. Changing Ownership Back

Return to original user:

```
exit  
sudo chown prabhat demo.txt
```

Ownership returns to prabhat.

6. Changing Both User and Group Together

Structure:

```
sudo chown USER:GROUP filename
```

Example:

```
sudo chown user2:developers demo.txt
```

Check:

```
ls -l demo.txt
```

Example result:

```
-rw-r--r-- 1 user2 developers demo.txt
```

Explanation:

- User owner updated
- Group owner updated

7. Changing Only the Group

Use colon before group name:

```
sudo chown :developers demo.txt
```

Check:

```
ls -l demo.txt
```

User remains the same, group changes.

8. Comparison

Command	Meaning
chown user2 file	change user owner
chown user2:group file	change user and group
chown :group file	change group only

PRACTICAL LABS WITH MULTI-USER TESTING

1. Ownership vs Permission Behaviour

Set ownership to user2:

```
sudo chown user2 demo.txt
```

Switch to user2:

```
su - user2
```

Try:

```
chmod 777 demo.txt
```

Works because owner can change permissions.

2. Non-Owner Cannot Change Permissions

Switch back to prabhat while owner is user2:

```
su - prabhat  
chmod 755 demo.txt
```

Expected:

```
Operation not permitted
```

WHY:

Ownership controls permission authority.

3. Recursive Ownership Change (Folders)

Create folder:

```
mkdir project  
touch project/file1.txt  
touch project/file2.txt
```

Change ownership recursively:

```
sudo chown -R user2 project
```

Explanation:

-R = recursive

Changes ownership of:

- folder
- all files inside

4. Checking Results

```
ls -l project
```

Files now belong to user2.

5. Safety Warning

Recursive chown is powerful.

Never run blindly on system folders like:

```
/etc  
/bin  
/usr
```

Wrong ownership can break Linux.

6. Practical Behaviour Test After Recursive Change

Switch to user2:

```
su - user2  
cd project
```

Edit files:

```
echo "edit" >> file1.txt
```

Works because ownership transferred.

7. Questions

Q1. Why does chown need sudo?

Changing ownership affects system security.
Only root can transfer file identity.

Q2. Difference Between chmod and chown?

Command	Controls
chmod	permissions (r w x)
chown	ownership (user/group)

Q3. Does chown Change Permissions?

No.

It only changes owner or group.

Q4. What Happens if Owner Changes?

New owner gains power to:

- edit permissions
- modify file
- manage access

8. Final Summary

Concept	Meaning
Owner	Primary controller of file
Group	Shared access group
Permissions	Access rules
chown	changes identity
chmod	changes rules